

Aprendizaje por Refuerzo Aproximaciones

Daniela Pucci de Farias

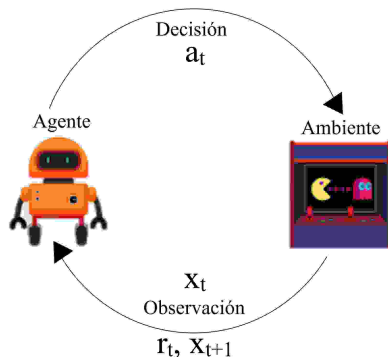
Universidad Torcuato Di Tella

June 2025

Clase 5

- ▶ La “Maldición de la Dimensionalidad”
 - ▶ MDPs de larga escala
 - ▶ Arquitecturas de aproximación
- ▶ Aplicación: Precificación de opciones
 - ▶ Teoría de opciones
 - ▶ Formulación MDP y modelaje de precios de activos
 - ▶ Simulación Monte Carlo

Revisión - MDPs y Aprendizaje por Refuerzo



► **Agente:**

- función valor $V(x)$ o $Q(x, a)$
- política $u(x)$

► **Ambiente:**

- recompensa $r_a(x)$
- transición $P_a(x, y)$

Revisión - MDPs y Aprendizaje por Refuerzo

- Función valor definida por la ecuación de Bellman:

$$\begin{aligned}V^*(x) &= \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\} \\Q^*(x, a) &= \left\{ r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a') \right\} \\V^*(x) &= \max_{a \in A} Q^*(x, a)\end{aligned}$$

- Resolvemos la ecuación de Bellman exactamente o con aprendizaje:

Soluciones Exactas	Aprendizaje
Iteración de valor	Simulación Monte Carlo (política fija)
Iteración de política	TD-Learning (política fija o variable)
	Q-Learning

- Calculamos la política óptima a partir de V^* o Q^* .

Q-Learning

1. Inicializar $\hat{Q}$ según alguna regla y hacemos $t = 1$.
2. Seleccionar acción a^i
3. Con observación (x^i, a^i, r^i, y^i) , actualizar

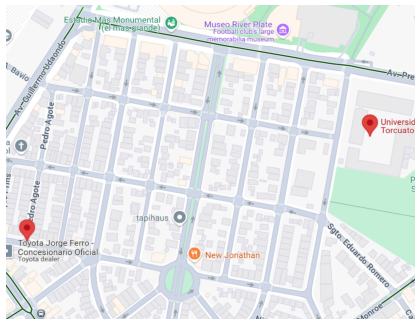
$$\hat{Q}(x^i, a^i) = \hat{Q}(x^i, a^i) + \gamma_i \left[r^i + \alpha \max_{a \in A} \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]$$

4. Hacer $t = t + 1$ y retornar al paso 2.

- Exploración vs explotación: por ejemplo, elegir acción ϵ -greedy.
- Tasa de aprendizaje γ_i ? Por ejemplo, $\gamma_i = 1/N_{x^i, a^i}$.

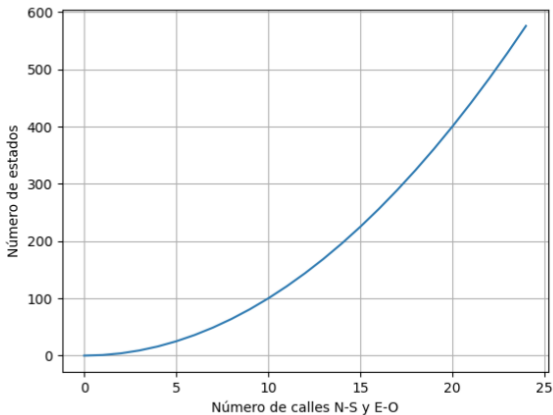
La “Maldición de la Dimensionalidad”

- ▶ Debemos almacenar V^* o Q^*
- ▶ Tabla con un valor por estado o par estado/acción.
- ▶ Cuánto espacio necesario?
- ▶ Ejemplo: Ruta casa a trabajo
 - ▶ N calles norte-sur, M calles este-oeste
 - ▶ Número de estados = $N \times M$
 - ▶ Número de estados/acciones = $N \times M \times 4$



La “Maldición de la Dimensionalidad”

- ▶ Asumiendo $M = N$ (mapa cuadrado):



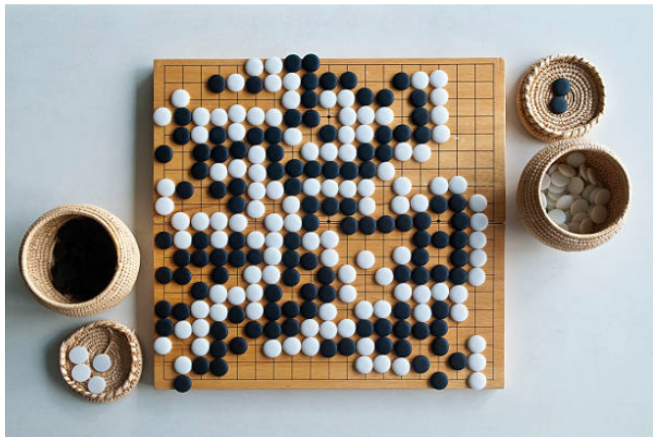
La “Maldición de la Dimensionalidad”

- ▶ Número de estados = N^2
- ▶ Crecimiento cuadrático en el número de estados con las dimensiones del grid
- ▶ No tan dramático, porque son pocas dimensiones
 - ▶ Variable de estado = (i,j)
 - ▶ Variable de acción en N, S, E, O
 - ▶ estado tiene 2 dimensiones, acción tiene 1
- ▶ Nuevo problema: a cada día, una de las cuadras puede estar cortada
 - ▶ Cómo modelar ese problema?
 - ▶ Cuántos estados?

La “Maldición de la Dimensionalidad”

- ▶ Cada cuadra c tiene una variable de estado con valores $\{?, \text{libre}, \text{cortada}\}$
- ▶ Tenemos una variable de estado extra para cada calle
- ▶ Estados posibles:
 - ▶ Si aún no hemos encontrado una calle cortada:
 - ▶ $(c_1, c_2, \dots, c_K), c_i \in \{?, \text{libre}\}$
 - ▶ 2^K estados
 - ▶ Si ya hemos encontrado una calle cortada:
 - ▶ $(c_1, c_2, \dots, c_n) = (\text{libre}, \text{libre}, \dots, \text{libre}, \text{cortada}, \text{libre}, \dots, \text{libre})$ porque suponemos que hay solo una calle cortada
 - ▶ K posibilidades
- ▶ **El número de estados crece exponencialmente con el número de variables de estado $\rightarrow$ no podemos almacenar la función valor**

Ejemplo: Go



Ejemplo: Go

Configuración y fuerza⁷

Versiones ↕	Hardwares ↕	Elo ↕	Partidos ↕
AlphaGo Fan	176 GPUs, distribuido	3.144	5:0 contra Fan Hui
AlphaGo Lee	48 TPUs , distribuido	3.739	4:1 contra Lee Sedol
AlphaGo Master	Una sola máquina con 4 TPU v2	4.858	60:0 contra jugadores profesionales; Cumbre del Futuro de Go
AlphaGo Zero	Una sola máquina con 4 TPUs ⁸ v2	5.185 ⁹	100:0 contra AlphaGo Lee 89:11 contra AlphaGo Master

Ejemplo: Go - Cuántos estados?

- ▶ Tablero de 19×19
 - ▶ $19 \times 19 = 361$ posiciones/variables de estado
 - ▶ cada posición puede tener 3 valores: blanco, negro, vacío
 - ▶ 3^{361} *estados*
 - ▶ pero no todas las configuraciones son válidas
 - ▶ aproximadamente $2.081681994 \times 10^{170}$ posiciones (2006)
 - ▶ en 2015, con avances en la potencia de computadores y nueve meses de procesamiento, John Tromp calculó el número de estados:

2081681993819799846
9947863334486277028
6522453884530548425
6394568209274196127
3801537852564845169
8519643907259916015
6281285460898883144
2712971531931755773
6620397247064840935

La “Maldición de la Dimensionalidad”

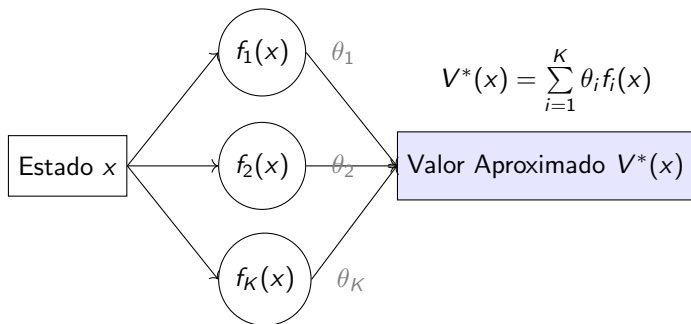
- ▶ En la práctica, número de estados es frecuentemente prohibitivo. No podemos calcular y almacenar la función valor.
- ▶ Solución: **generalización**
 - ▶ Generalizar lo que sabemos sobre la función valor y o política a otros estados y acciones
- ▶ Aproximamos la función valor y/o la función política con una **forma paramétrica**:

$$\begin{aligned}V^*(x) &\approx f(x, \theta_V), \quad \theta_V \in \mathbb{R}^K \\Q^*(x, a) &\approx f(x, a, \theta_Q), \quad \theta_Q \in \mathbb{R}^K \\u^*(x, a) &\approx f(x, a, \theta_u), \quad \theta_u \in \mathbb{R}^K\end{aligned}$$

- ▶ Es común considerar políticas randomizadas $u(x, a) =$ probabilidad de cada acción a en el estado x cuando aproximamos la función política.

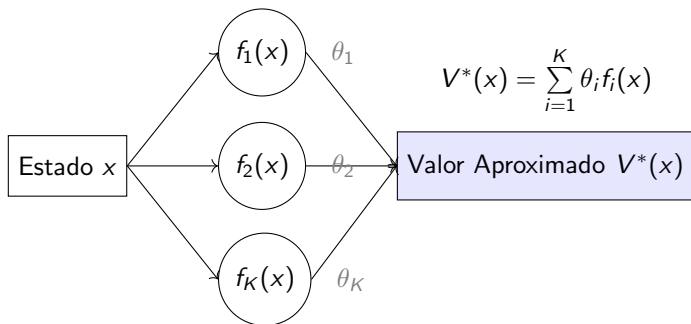
Aproximaciones Lineales

- ▶ Queremos aproximar el valor de un estado $V(x)$ sin usar una tabla
- ▶ Usamos una combinación de funciones conocidas ("funciones básicas"):



- ▶ Esto se llama **aproximación lineal en los parámetros** (aunque $f_i(x)$ puede ser no lineal en x)

Ejemplos de funciones básicas $f_i(x)$

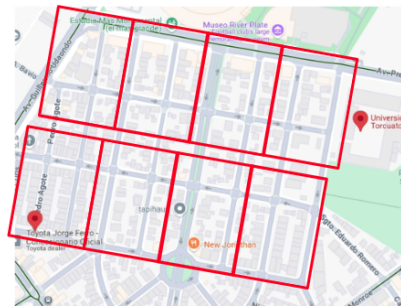


- ▶ Valores individuales de componentes del estado : $f_i(x) = x_i$
- ▶ Términos polinomiales: $x_1^2, x_1x_2, \dots$
- ▶ Otras funciones no lineales: $\sin(x_1), \log(x_2 + 1), \dots$
- ▶ **Agregación de estados:** dividir el espacio en regiones y usar 1 o 0 según a cuál pertenece x

Agregación de Estados

$$f_i(x) = \begin{cases} 1 & \text{si } x \in S_i \\ 0 & \text{si } x \notin S_i \end{cases}$$

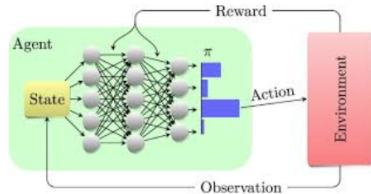
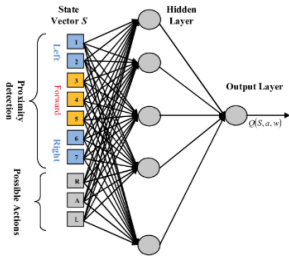
- ▶ $V^*(x) \approx V(\text{zona}(x))$
- ▶ Buenas propiedades de convergencia
- ▶ Posibilidad de particionar espacio de estados progresivamente



Aproximaciones No Lineales

$$V^*(x) \approx f(x, \theta)$$

- $f(x, \theta)$ es no lineal en θ
- Ejemplo: redes neuronales



Aprendizaje Por Refuerzo Con Aproximaciones

- ▶ Aproximar $V^*(x) \approx f(x, \theta)$ tiene aspectos similares a aprendizaje supervisado:
 - ▶ Tenemos pares (x, y) y elegimos θ para que $\hat{y} = f(x, \theta)$ se parezca a y .
 - ▶ Pérdida típica: $\min_{\theta} \sum (y - f(x, \theta))^2$.
- ▶ Pero en RL, no hay “label” y .
- ▶ El verdadero valor $V^*(x)$ está definido implícitamente por la ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}, \quad \forall x.$$

- ▶ En vez de ajustar contra un “label”, tratamos de encontrar un valor de θ de tal forma que $V(x, \theta)$ sea una solución aproximada a la ecuación de Bellman:

$$\min_{\theta} \sum_x \underbrace{p(x)}_{\text{“peso” del estado}} \underbrace{\left[\max_a \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V(y, \theta) \right\} - V(x, \theta) \right]^2}_{\text{residuo de Bellman}}$$

Aprendizaje por Refuerzo con Aproximaciones

- ▶ Queremos resolver:

$$\min_{\theta} \sum_x \underbrace{p(x)}_{\text{"peso" del estado}} \underbrace{\left[\max_a \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V(y, \theta) \right\} - V(x, \theta) \right]^2}_{\text{residuo de Bellman}}$$

- ▶ Con el espacio de estados demasiado largo, mismo conociendo r y P no podemos calcular el error exactamente, debemos tomar muestras
- ▶ Aplicamos versiones de algoritmos de aprendizaje por refuerzo
 - ▶ Convergencia no siempre garantizada
 - ▶ Requiere técnicas de estabilización: normalización, redes objetivo, replay buffers, etc.
 - ▶ Resultados sensibles a la arquitectura y los hiperparámetros
- ▶ Ejemplos con redes neuronales:
 - ▶ **DQN (Deep Q-Networks)**: $Q(x, a; \theta) = \text{output de red neuronal}$
 - ▶ **MuZero**: redes neuronales para modelo latente + valor + política

Aplicación: Valuación de Opciones Americanas

”Opciones son instrumentos derivados que otorgan el derecho -y no la obligación- a comprar o vender activos a un precio determinado en el contrato -denominado precio de ejercicio o strike- hasta una fecha establecida, es decir el vencimiento del contrato.”

- ▶ Derecho a comprar el activo = opción call
- ▶ Derecho a vender el activo = opción put
- ▶ Ejercicio del derecho solo en la fecha de vencimiento = **opción europea**
- ▶ Ejercicio del derecho hasta la fecha de vencimiento = **opción americana**

Aplicación: Valuación de Opciones Americanas

Consideramos una opción put americana, con fecha de vencimiento T y precio de ejercicio K .

- ▶ Cuál debe ser el precio de la opción en $t = 0$?
- ▶ Cuándo se debe ejercer la opción?

Modelo MDP para Valuación de Opciones

- ▶ Debemos definir (S, A, P, r)
- ▶ Estado $s_t =$ precio del activo en el periodo t
- ▶ Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- ▶ Recompensa $r_a(s)$:
 - ▶ $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)
 - ▶ Cuánto ganamos si ejercemos la opción en tiempo t ?
 - ▶ Compramos la acción a precio $s_t =$ precio del activo
 - ▶ Vendemos a precio K
 - ▶ $r_1(s) = K - s$
- ▶ Debemos especificar s_t , o más generalmente la evolución de precio del activo para terminar de definir el problema. . .

Modelo de Precios Para Valuación de Opciones

- ▶ Recordemos el modelo lognormal para precios:

$$s_{t+1} = s_t e^{r - \sigma^2/2 + \sigma \epsilon_t}$$

donde

- ▶ ϵ_t , $t = 0, 1, \dots$ son variables aleatorias independientes y siguen la distribución normal estándar ($\epsilon_t \sim N(0, 1)$)
- ▶ el rendimiento r y volatilidad σ son expresados con respecto a la duración de cada período de decisión.
- ▶ Para calcular el valor de la opción, podríamos encontrar la estrategia óptima de ejercicio y evaluarla.
- ▶ Esa solución, basada en el valor esperado, no considera el riesgo de quién vende la opción.
- ▶ Podemos aplicar la metodología modificando el modelo de precios para que sea **neutral al riesgo**.
- ▶ Si el activo no paga dividendos, el modelo neutral al riesgo es simplemente:

$$s_{t+1} = s_t e^{r_f - \sigma^2/2 + \sigma \epsilon_t}$$

donde r_f es la tasa de interés libre de riesgo.

Modelo MDP para Valuación de Opciones

- ▶ Debemos definir (S, A, P, r)
- ▶ Estado s_t = precio del activo en el periodo t
- ▶ Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- ▶ Recompensa $r_a(s)$:
 - ▶ $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)
 - ▶ Cuánto ganamos si ejercemos la opción en tiempo t ?
 - ▶ Compramos la acción a precio s_t = precio del activo
 - ▶ Vendemos a precio K
 - ▶ $r_1(s) = K - s$

- ▶ Transición:

$$s_{t+1} = s_t e^{r_f - \sigma^2/2 + \sigma \epsilon_t}$$

- ▶ factor de descuento $\alpha = e^{-r_f}$

Ecuaciones de Bellman

- ▶ Cómo definir la función valor? Analizaremos la función Q^* .
- ▶ El valor de ejercer la acción en cualquier periodo t es conocido:

$$Q_t^*(s, 1) = K - s, \quad \forall t$$

- ▶ Podemos escribir las ecuaciones para el valor de no ejercer en el periodo actual ($a = 0$) en cada periodo $t < T$:

$$\begin{aligned} Q_t^*(s, 0) &= \alpha E \left[\max \left(Q_{t+1}^*(s_{t+1}, 0), Q^*(s_{t+1}, 1) \right) \mid s_t = s \right], \\ &= \alpha E \left[\max \left(Q_{t+1}^*(s_{t+1}, 0), K - s_{t+1} \right) \mid s_t = s \right] \end{aligned}$$

- ▶ En el último periodo, si no ejercitamos la opción, su valor es cero:

$$Q_T^*(s, 0) = 0$$

Problemas de Parada Óptima

- ▶ Ecuaciones de Bellman dependen solo de $Q_t^*(s, 0)$:

$$\begin{aligned}Q_t^*(s, 0) &= \alpha E \left[\max \left(Q_{t+1}^*(s_{t+1}, 0), K - s_{t+1} \right) \mid s_t = s \right] \\Q_T^*(s, 0) &= 0\end{aligned}$$

- ▶ Esa clase de MDPs es conocida como problemas de parada óptima.
- ▶ Resolviendo con aprendizaje por refuerzo:
 - ▶ generamos trayectorias $(s_t^i, t = 0, 1, \dots)$, $i = 1, 2, \dots, N$, siguiendo $s_{t+1}^i = s_t^i e^{r_f - \sigma^2/2 + \sigma \epsilon_t^i}$
 - ▶ aplicamos métodos de aprendizaje por refuerzo para resolver las ecuaciones de Bellman (simulación Monte Carlo, TD-Learning, versiones de iteración de valor e iteración de políticas,...)
 - ▶ vamos a implementar el método Monte Carlo con Python

Complejidad del MDP

- ▶ Con el modelo que estamos utilizando para el precio s_t del activo, tenemos solo una variable de estado/dimensión, que es el propio precio.
- ▶ En la práctica, puede ser necesario o ventajoso incorporar muchas otras variables de estado:
 - ▶ volatilidad σ y/o tasa de interés r_f pueden ser procesos que varían en el tiempo
 - ▶ pueden depender de otros factores
 - ▶ activo puede pagar dividendos
 - ▶ opción puede ser mucho más compleja y recompensa puede estar basada en distintas condiciones
- ▶ nuestro modelo asume que precios tienen valores continuos
 - ▶ número infinito de estados

Aproximaciones Para Q^*

- ▶ Mismo en el caso más simple de una variable de estado, aproximaciones son necesarias.
- ▶ Estrategia popular en la práctica (Longstaff y Schwartz):
 - ▶ basada en simulación Monte Carlo y recursión
 - ▶ aproximación con arquitectura linear:

$$Q_t^*(s, 0) \approx \sum_{i=1}^K f^i(s) \theta_t^i$$

- ▶ por ejemplo, $f_i(s) = s^i$ (aproximación polinomial)

Aproximación con Simulación Monte Carlo

1. Generamos N trayectorias $(s_0^i, s_1^i, \dots, s_T^i), i = 1, 2, \dots, N$.
2. En $t = T - 1$, resolvemos:

$$\theta_{T-1} = \arg \min_{\theta \in \mathbb{R}^K} \sum_{i=1}^N [\Phi(s_{T-1}^i, \theta) - \alpha \max(K - s_T^i, 0)]^2.$$

3. Para $t = T - 2, T - 3, \dots, 0$, resolvemos:

$$\theta_t = \arg \min_{\theta \in \mathbb{R}^K} \sum_{i=1}^N [\Phi(s_t^i, \theta) - \alpha \max(K - s_{t+1}^i, \Phi(s_{t+1}^i, \theta_{t+1}))]^2$$

donde $\Phi(s, \theta) = \sum_{j=1}^K \phi^j(s) \theta^j$.

4. Calculamos la política codiciosa u^Φ

$$u_t^\Phi(s) = \begin{cases} 1 & \text{si } K - s > \Phi(s, \theta_t) \\ 0 & \text{si } K - s \leq \Phi(s, \theta_t) \end{cases}$$

5. Estimamos el valor de la opción como el valor de la política u^Φ :

$$\frac{1}{N} \sum_{i=1}^N \alpha^{T_i} \max(K - s_{T_i}^i, 0)$$

Conclusiones

- ▶ Paradigma de MDP es poderoso y suficientemente flexible para modelar una amplia variedad de aplicaciones en distintos campos.
- ▶ Combinado con aprendizaje, potencial para encontrar soluciones competitivas.
- ▶ Aproximaciones a la función valor y/o política permite tratar problemas de alta complejidad y muchas dimensiones.
- ▶ Desafíos en la implementación estable y correcta de los algoritmos.
- ▶ Complejidad del abordaje debe ser acorde a los datos disponibles.
- ▶ Potencialmente grandes beneficios con la adopción de redes neuronales